

Coding Attention Mechanisms

Chapter 3: The Core of the Transformer Architecture

Graduate Course on Deep Learning

May 18, 2026

Instituto Politécnico Nacional (IPN)

The Motivation for Attention

- **Problem with RNNs:** Difficulty in capturing long-range dependencies due to the vanishing gradient problem and sequential bottleneck.
- **The Attention Concept:** Allows the model to focus on different parts of the input sequence regardless of their distance.
- **Key Insight:** Instead of a fixed-size hidden state, we compute a weighted sum of all input tokens' representations.

[Image of attention mechanism in transformer]

Scaled Dot-Product Attention: The Math

The core operation involves three learnable matrices: **Query (Q)**, **Key (K)**, and **Value (V)**.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (1)$$

- **Dot-Product (QK^T)**: Measures similarity between queries and keys.
- **Scaling ($1/\sqrt{d_k}$)**: Prevents the softmax from entering regions with extremely small gradients when d_k is large.
- **Softmax**: Normalizes scores into probabilities (weights summing to 1).

Self-Attention vs. Cross-Attention

Self-Attention:

- Q, K, V all originate from the same input sequence.
- Used in Encoders and Decoders to relate different positions of the same sequence.

Cross-Attention:

- Q comes from one sequence (e.g., decoder), while K and V come from another (e.g., encoder output).
- Essential for tasks like Machine Translation.

Causal Attention: Masking the Future

In GPT-like models (Decoder-only), a token should only attend to previous tokens.

- **Implementation:** Apply a triangular mask (lower triangular matrix) to the scores before the softmax.
- **Value:** Future positions are set to $-\infty$, resulting in 0 probability after softmax.

```
mask = torch.tril(torch.ones(seq_len, seq_len)
                    )
masked_scores = scores.masked_fill(mask == 0,
                                    float('-inf'))
```

Multi-Head Attention (MHA)

Why use multiple heads?

- Allows the model to jointly attend to information from different representation subspaces at different positions.
- **Process:**
 1. Split Q, K, V into h heads.
 2. Perform attention in parallel.
 3. Concatenate the results and apply a linear projection.

[Image of multi-head attention architecture]

Computational Efficiency and Complexity

- **Self-Attention Complexity:** $O(L^2 \cdot d)$, where L is sequence length and d is embedding dimension.
- **Comparison:**
 - RNNs are $O(L \cdot d^2)$ but sequential.
 - Transformers are parallelizable but quadratic with respect to length.
- This quadratic growth is why extending the **Context Window** is a major research challenge in LLMs.

From Math to Code: PyTorch Structure

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_in, d_out, context_len,
                  dropout, num_heads):
        super().__init__()
        self.W_query = nn.Linear(d_in, d_out, bias=
                                   False)
        self.W_key = nn.Linear(d_in, d_out, bias=False
                                )
        self.W_value = nn.Linear(d_in, d_out, bias=
                                   False)
        # ... rest of the logic
```

Crucial: Biases are often omitted in Q, K, V linear layers in modern implementations to simplify the geometry of the projection.

Next Chapter: Implementing a GPT Model from Scratch